



Medication Reminder Chatbot

Label-Aware Medical Assistant using RAG Architecture

Table of Contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [Installation & Setup](#)
 4. [Component Documentation](#)
 5. [Usage Guide](#)
 6. [API Reference](#)
 7. [Troubleshooting](#)
-

Project Overview

The **Medication Reminder Chatbot** is an intelligent healthcare assistant that leverages Retrieval Augmented Generation (RAG) to provide accurate, label-based information about medications. Built with a focus on safety and accuracy, it retrieves information directly from FDA drug labels stored in a vector database.

Key Features



Intelligent Q&A

Ask natural language questions about medications and receive accurate answers sourced directly from official drug labels.



Smart Reminders

Automatically generate medication reminder schedules based on dosage instructions from the drug label.



Multi-Chat Interface

Manage multiple conversations simultaneously with separate context for different medications.

Safety Guardrails

Built-in validation ensures queries are safe and responses stay within label information boundaries.

Autonomous Ingestion

Automatically processes and indexes drug label data on first run without manual intervention.

Vector Search

ChromaDB-powered semantic search finds relevant information quickly across large datasets.

Technology Stack

- **Streamlit** - Web interface and UI components
- **LangChain** - RAG framework and chain orchestration
- **ChromaDB** - Vector database for semantic search
- **Ollama (Llama 3 8B)** - Local LLM for generation
- **Nomic Embed Text** - Embedding model
- **Python 3.8+** - Core programming language

Design Philosophy

The system prioritizes:

- **Accuracy** - Information sourced only from official drug labels
 - **Safety** - Guardrails prevent medical advice generation
 - **Transparency** - Clear indication when information is unavailable
 - **Usability** - Intuitive chat interface with minimal learning curve
 - **Privacy** - All processing happens locally with Ollama
-

System Architecture

Project Structure

```
medication_bot/
|
├── app.py           # Streamlit UI and interaction logic
├── agents.py        # Ingestion, QA, Reminder agents
├── vectorstore.py   # ChromaDB setup and helpers
├── guardrails.py    # Input validation and safety checks
├── evals.py         # Basic evaluation utilities
├── ingest.py        # Manual ingestion script
├── sample.txt       # Drug label data (openFDA-style JSON)
├── chroma_poc/      # Persisted vector database
├── requirements.txt # Dependencies
└── README.md        # Project documentation
```

RAG Pipeline Architecture

1. Data Ingestion Phase

```
FDA Drug Labels (JSON)
  ↓
Document Chunking (800 chars)
  ↓
Embedding Generation (Nomic Embed)
  ↓
ChromaDB Storage (Persistent)
```

2. Query Processing Phase

```
User Question
  ↓
Query Embedding
  ↓
Semantic Search (Top 3)
  ↓
Context Retrieval
```

3. Response Generation Phase

Retrieved Context + User Query



Prompt Construction



LLM Generation (Llama 3)



Grounded Response

Agent Architecture

Ingestion Agent

Responsibility: Processes FDA JSON records into structured documents

Input: Raw FDA drug label JSON

Output: List of Document objects with metadata

Key Operations:

- Extracts 11 different label sections
- Adds product and section metadata
- Chunks text into optimal sizes (800 chars)
- Preserves product identification

Q&A Agent

Responsibility: Answers questions using retrieved label context

Input: Product name + User question

Output: Natural language response

Key Operations:

- Retrieves top 3 relevant chunks
- Constructs safety-constrained prompt
- Generates answer via LLM
- Returns "not available" if no context found

Safety Rules:

- Answer ONLY from drug label context
- Do NOT add medical advice
- Clearly state when information is missing

Reminder Agent

Responsibility: Generates medication schedules from dosage info

Input: Product name

Output: Structured JSON reminder schedule

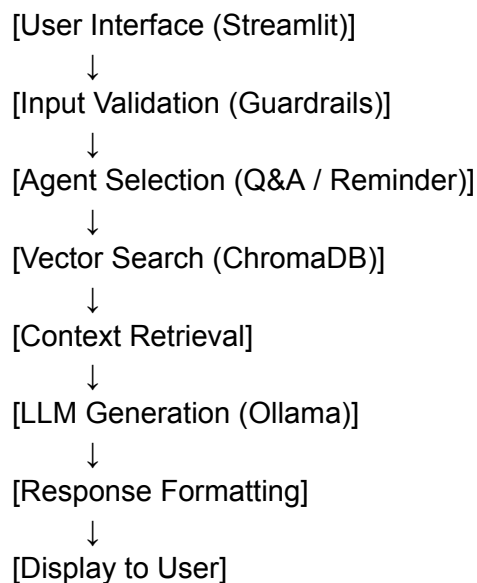
Key Operations:

- Retrieves dosage_and_administration sections
- Prompts LLM for strict JSON output
- Parses and validates response
- Handles malformed JSON gracefully

Output Schema:

```
{
  "product": "Medication Name",
  "schedule": [
    {
      "time": "Morning",
      "instruction": "Take 1 tablet with food"
    }
  ],
  "notes": "Additional instructions..."
}
```

Data Flow Diagram



Installation & Setup

Prerequisites

Before installing, ensure you have:

1. **Python 3.8 or higher** installed
2. **Ollama** installed and running locally
3. **Git** for cloning the repository
4. **8GB+ RAM** recommended for local LLM

Step 1: Install Ollama

macOS/Linux:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Windows:

Download installer from: <https://ollama.ai>

Pull Required Models:

```
ollama pull llama3:8b
ollama pull nomic-embed-text
```

Verify Installation:

```
ollama list
# Should show both models
```

Step 2: Clone Repository

```
git clone <repository-url>
cd medication_bot
```

Step 3: Create Virtual Environment (Recommended)

```
# Create virtual environment
python -m venv venv

# Activate virtual environment
# On macOS/Linux:
```

```
source venv/bin/activate
# On Windows:
venv\Scripts\activate
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

Dependencies installed:

- streamlit - Web UI framework
- langchain - RAG framework
- langchain-community - Community integrations
- chromadb - Vector database
- ollama - Python client for Ollama

Step 5: Prepare Data

Ensure **sample.txt** contains FDA drug label data in JSON format.

Expected Format:

```
[
  {
    "id": "unique_id",
    "openfda": {
      "brand_name": ["Product Name"]
    },
    "indications_and_usage": ["Usage text..."],
    "warnings_and_cautions": ["Warning text..."],
    "dosage_and_administration": ["Dosage text..."],
    ...
  }
]
```

Step 6: Run the Application

```
streamlit run app.py
```

Access the app at: <http://localhost:8501>

First Run Behavior

⚙️ **Automatic Ingestion:** On first run, the system detects an empty database and automatically:

1. Reads sample.txt
2. Processes all records through ingestion agent
3. Generates embeddings
4. Stores in ChromaDB (./chroma_poc/)
5. Persists database to disk

This process takes 2-5 minutes depending on data size.

✅ **Subsequent Runs:** The system detects existing data and skips ingestion, loading instantly.

Verification

After successful setup, you should see:

- Streamlit UI with "Medication Reminder Chatbot" title
 - Dropdown with available medications
 - Welcome message in chat
 - No error messages in console
-

Component Documentation

1. app.py - Main Application

Purpose: Streamlit-based user interface providing chat functionality and medication management.

Key Features

Multi-Chat Support

- Create unlimited independent chat sessions
- Each chat maintains separate conversation history
- Switch between chats via sidebar
- Isolated product selection per chat

Product Selection Interface

- Dynamic dropdown populated from database
- Shows all available medications
- Validates product before querying

- Clear product labeling in chat

Action Buttons

- **Generate Reminder** - Quick access to reminder generation
- **Change Drug** - Switch medication without resetting chat
- **Quit Chat** - Reset current conversation

Session State Management

```
st.session_state.chats = {  
    "Chat 1": {  
        "messages": [...],  
        "product": "Product Name"  
    }  
}
```

User Commands

Command	Action	Example
reminder	Generate medication schedule	"reminder"
/change	Change selected medication	"/change"
/quit	Reset current chat	"/quit"
Regular text	Ask question about medication	"What are the side effects?"

UI Flow

1. **Initial State** - Welcome message, product selection dropdown
2. **Product Selected** - Action buttons appear, chat input enabled
3. **User Input** - Message processed through agents
4. **Response Display** - Answer shown in chat with proper formatting
5. **History Maintained** - All messages persist in session state

Error Handling

The application handles:

- Empty database → Error message with instructions
- Invalid product → Validation error
- Empty questions → Input validation error
- Agent failures → Graceful error display in chat

Code Structure

```
# Session initialization
if 'chats' not in st.session_state:
    st.session_state.chats = {}

# Sidebar chat management
st.sidebar.title("Chats")
for name in st.session_state.chats.keys():
    if st.sidebar.button(name):
        st.session_state.current_chat = name

# Main chat interface
if product is None:
    # Show product selection
else:
    # Show chat interface with buttons
```

2. agents.py - AI Agent Logic

Purpose: Core intelligence layer containing ingestion, Q&A, and reminder agents.

Helper Functions

extract_list_field(record, key)

Parameters:

- **record** (dict) - FDA JSON record
- **key** (str) - Field name to extract

Returns: String (joined list items or single value)

Purpose: Safely extracts list fields from JSON, handling both list and string types.

Example

warnings = extract_list_field(record, "warnings_and_cautions")

Returns: "Warning 1\nWarning 2\nWarning 3"

chunk_text(text, max_chars=800)

Parameters:

- **text** (str) - Text to chunk
- **max_chars** (int) - Maximum characters per chunk

Returns: List of text chunks

Purpose: Splits long text into optimal chunks for embedding (800 chars default).

Why 800 chars?

- Balances context preservation with embedding quality
- Prevents token limit issues
- Maintains semantic coherence

Autonomous Ingestion Pipeline

autonomous_ingestion()

Purpose: Automatically checks database and performs ingestion if empty.

Flow:

1. Check if vectorstore is empty
2. If not empty → Skip ingestion
3. If empty → Load sample.txt
4. Process each record through ingestion_agent()
5. Chunk all documents
6. Generate embeddings
7. Store in ChromaDB
8. Persist to disk

Returns: ChromaDB vectorstore instance

Console Output:

- ⚙️ Vector DB empty. Running ingestion agent...
- ✅ Autonomous ingestion completed. Chunks stored: 245

Ingestion Agent

ingestion_agent(record)

Purpose: Converts FDA JSON record into multiple structured documents.

Input: Single FDA drug label JSON object

Output: List of Document objects (one per section)

Processed Sections:

1. **indications_and_usage** - What the drug treats
2. **warnings_and_cautions** - Safety warnings
3. **active_ingredients** - Chemical composition
4. **boxed_warning** - Critical safety information (black box)

5. **dosage_and_administration** - How to take the medication
6. **adverse_reactions** - Side effects
7. **drug_interactions** - Interaction with other drugs
8. **contraindications** - When not to use
9. **keep_out_of_reach_of_children** - Child safety warnings
10. **pregnancy** - Pregnancy-related information
11. **pediatric_use** - Children usage information

Document Structure:

```
Document(  
    page_content=f"PRODUCT: {brand}\nSECTION: {section_name}\n\n{text}",  
    metadata={  
        "product": brand,  
        "section": section_name,  
        "id": product_id,  
        "chunk": chunk_index  
    }  
)
```

Metadata Usage:

- **product** - Enables product-specific filtering
- **section** - Allows section-targeted retrieval
- **id** - Tracks source record
- **chunk** - Identifies chunk position

Q&A Agent

ask_drug(product, question)

Purpose: Answers questions using retrieved label context.

Parameters:

- **product** (str) - Medication name
- **question** (str) - User's question

Returns: String response from LLM

Process:

1. Create retriever with product filter
2. Retrieve top 3 relevant chunks
3. Check if documents found
4. Construct prompt with context and safety rules
5. Invoke LLM
6. Return response

Retrieval Configuration:

```
retriever = vectorstore.as_retriever(  
    search_kwargs={  
        "k": 3, # Top 3 results  
        "filter": {"product": product} # Product-specific  
    }  
)
```

Prompt Template:

You are a medication assistant.

RULES:

- Answer ONLY from the drug label context.
- Do NOT add medical advice.
- If information is missing, say: "Information not available in the label."

Drug Label Context:

{context}

Question:

{question}

Safety Measures:

- Strict context-only answers
- No medical advice generation
- Clear unavailability messages
- Product-scoped retrieval

Reminder Agent

reminder_agent(product)

Purpose: Generates structured medication schedules.

Parameters:

- **product** (str) - Medication name

Returns: Dict with parsed JSON or error message

Process:

1. Retrieve dosage_and_administration sections
2. Check for dosage information

3. Construct strict JSON prompt
4. Invoke LLM
5. Parse response with fallback handling
6. Return structured data

Retrieval Configuration:

```
retriever = vectorstore.as_retriever(
    search_kwargs={
        "k": 3,
        "filter": {
            "$and": [
                {"product": product},
                {"section": "dosage_and_administration"}
            ]
        }
    }
)
```

Prompt Requirements:

- MUST return ONLY raw JSON
- NO markdown formatting
- NO explanations before/after
- Output starts with { and ends with }

Output Schema:

```
{
  "product": "string",
  "schedule": [
    {
      "time": "string",
      "instruction": "string"
    }
  ],
  "notes": "string"
}
```

parse_or_fallback(text)

Purpose: Robust JSON parsing with error handling.

Returns:

Success case

```
{
```

```
"type": "json",  
"content": {parsed_json}  
}
```

```
# Failure case
```

```
{  
  "type": "raw",  
  "content": "raw text"  
}
```

Parsing Strategy:

1. Extract JSON block using regex
 2. Attempt JSON parsing
 3. Return parsed data if successful
 4. Return raw text if parsing fails
 5. Gracefully handle missing JSON
-

3. vectorstore.py - Database Management

Purpose: ChromaDB initialization and helper functions for vector database operations.

Core Functions

get_vectorstore()

Purpose: Initializes and returns ChromaDB instance.

Returns: Chroma vectorstore object

Configuration:

```
embeddings = OllamaEmbeddings(model="nomic-embed-text")  
return Chroma(  
    persist_directory="./chroma_poc",  
    embedding_function=embeddings  
)
```

Key Details:

- **Embedding Model:** nomic-embed-text (768 dimensions)
- **Storage Location:** ./chroma_poc/ directory
- **Persistence:** Automatic on write operations
- **Thread-Safe:** Safe for concurrent access

Usage:

```
vectorstore = get_vectorstore()  
# Use for retrieval, querying, etc.
```

get_available_products(vectorstore)

Purpose: Retrieves unique product names from database metadata.

Parameters:

- **vectorstore** - ChromaDB instance

Returns: Sorted list of unique product names

Implementation:

```
data = vectorstore._collection.get(include=["metadatas"])  
products = sorted(  
    set(  
        m.get("product")  
        for m in data["metadatas"]  
        if m and "product" in m  
    )  
)  
return products
```

Features:

- Extracts from metadata only (efficient)
- Removes duplicates via set()
- Alphabetically sorted
- Handles empty database gracefully

Usage:

```
products = get_available_products(vectorstore)  
# Returns: ["Aspirin", "Ibuprofen", "Lisinopril", ...]
```

is_db_empty(vectorstore)

Purpose: Checks if database needs initialization.

Parameters:

- **vectorstore** - ChromaDB instance

Returns: Boolean (True if empty, False if populated)

Implementation:

```
try:
    return vectorstore._collection.count() == 0
except:
    return True # Assume empty on error
```

Use Case:

```
if is_db_empty(vectorstore):
    print("Running ingestion...")
    autonomous_ingestion()
else:
    print("Database already populated")
```

Database Schema

Collection Structure:

- **Documents:** Text chunks from drug labels
- **Embeddings:** 768-dimensional vectors (nomic-embed-text)
- **Metadata:** Product, section, ID, chunk index
- **IDs:** Auto-generated unique identifiers

Metadata Schema:

```
{
  "product": "Product Name",
  "section": "indications_and_usage",
  "id": "record_id",
  "chunk": 0
}
```

Storage Details

Directory Structure:

```
./chroma_poc/
├── chroma.sqlite3  # SQLite database file
└── [embedding files] # Binary embedding data
```

Size Considerations:

- ~5-10 MB per 100 documents
 - Embeddings dominate storage
 - Scales linearly with documents
-

4. guardrails.py - Input Validation

Purpose: Provides safety checks to prevent invalid operations and maintain system integrity.

Validation Functions

validate_product(product, valid_products)

Purpose: Ensures selected product exists in database.

Parameters:

- `product` (str) - Product name to validate
- `valid_products` (list) - List of valid product names

Raises: ValueError if product invalid

Implementation:

```
def validate_product(product, valid_products):  
    if product not in valid_products:  
        raise ValueError("Invalid product selected")
```

Usage:

```
try:  
    validate_product("Aspirin", products)  
    # Proceed with query  
except ValueError as e:  
    st.error(str(e))
```

Why Important:

- Prevents queries to non-existent products
- Avoids empty retrieval results
- Maintains data integrity
- Provides clear error messages

validate_question(question)

Purpose: Prevents empty or whitespace-only queries.

Parameters:

- `question` (str) - User's question to validate

Raises: ValueError if question empty

Implementation:

```
def validate_question(question):  
    if not question or not question.strip():  
        raise ValueError("Question cannot be empty")
```

Validation Checks:

- Not None
- Not empty string
- Not only whitespace
- Has actual content

Usage:

```
try:  
    validate_question(user_input)  
    answer = ask_drug(product, user_input)  
except ValueError as e:  
    st.warning(str(e))
```

Guardrail Philosophy

The guardrails follow these principles:

1. **Fail Fast** - Validate early before expensive operations
2. **Clear Errors** - Descriptive messages for debugging
3. **User-Friendly** - Error messages guide users to fix issues
4. **Minimal Overhead** - Simple, efficient checks
5. **Extensible** - Easy to add new validators

Future Enhancements

Potential additional guardrails:

- Question length limits (prevent token overflow)
- Profanity filtering
- Medical advice detection
- Rate limiting per session

- Input sanitization for SQL injection
-

5. evals.py - Evaluation Utilities

Purpose: Basic quality checks for retrieval and generation outputs.

Evaluation Functions

`eval_retrieval(docs)`

Purpose: Verifies that documents were retrieved.

Parameters:

- `docs` (list) - List of retrieved documents

Returns: Boolean (True if documents found, False otherwise)

Implementation:

```
def eval_retrieval(docs):  
    return len(docs) > 0
```

Usage:

```
docs = retriever.invoke(question)  
if eval_retrieval(docs):  
    print("✅ Retrieval successful")  
else:  
    print("❌ No documents retrieved")
```

What It Checks:

- Documents were returned
- List is not empty
- Retrieval didn't fail silently

`eval_reminder_json(output)`

Purpose: Validates reminder JSON structure.

Parameters:

- `output` (str) - Raw LLM output

Returns: Boolean (True if valid JSON with schedule, False otherwise)

Implementation:

```
def eval_reminder_json(output):  
    try:  
        data = json.loads(output)  
        return "schedule" in data  
    except Exception:  
        return False
```

Validation Criteria:

- Valid JSON format
- Contains "schedule" field
- Parseable structure

Usage:

```
reminder = reminder_agent(product)  
if eval_reminder_json(reminder):  
    print("✅ Valid reminder generated")  
else:  
    print("❌ Invalid reminder format")
```

Evaluation Strategy

Current Approach:

- Simple, binary checks
- Fast execution
- Minimal dependencies
- Basic quality gates

Metrics Not Covered:

- Answer relevance
- Factual accuracy
- Response completeness
- User satisfaction
- Context appropriateness

Future Evaluation Enhancements

Potential Additions:

1. **Retrieval Quality**
 - Precision@K metrics

- Relevance scoring
- Coverage analysis
- 2. **Generation Quality**
 - BLEU/ROUGE scores
 - Factual consistency checks
 - Hallucination detection
- 3. **End-to-End Testing**
 - Question-answer pairs
 - Expected output matching
 - Regression testing
- 4. **User Experience**
 - Response time tracking
 - Error rate monitoring
 - User feedback collection

Example Advanced Eval:

```
def eval_answer_quality(question, answer, context):
    """Check if answer is grounded in context"""
    # Use NLI model to verify entailment
    # Check for hallucinations
    # Measure relevance to question
    pass
```

6. ingest.py - Manual Ingestion Script

Purpose: Standalone script for manually processing and indexing drug label data.

Overview

While `autonomous_ingestion()` in `agents.py` handles ingestion automatically on app startup, this script provides:

- Manual control over ingestion timing
- Debugging and testing capabilities
- One-time database population
- Data verification before app launch

Script Flow

1. Load JSON records from `sample.txt`
2. Process each record through `ingestion_agent()`
3. Collect all generated documents

4. Generate embeddings using nomic-embed-text
5. Store in ChromaDB
6. Persist database to disk

Execution

python ingest.py

Console Output:

 Loaded 5 records
 Generated 245 documents
 Ingestion completed and ChromaDB persisted

Code Structure

```
# Load data
with open("sample.txt", "r", encoding="utf-8") as f:
    records = json.load(f)

# Process through agent
all_documents = []
for record in records:
    docs = ingestion_agent(record)
    all_documents.extend(docs)

# Build vectorstore
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(
    documents=all_documents,
    embedding=embeddings,
    persist_directory="./chroma_poc"
)

# Persist
vectorstore.persist()
```

When to Use

Use **ingest.py** when:

- Testing new data formats
- Debugging ingestion issues
- Pre-populating database before deployment
- Verifying data quality

- Batch processing large datasets

Use **autonomous_ingestion()** when:

- Normal app usage
- Development workflow
- Automatic initialization preferred
- First-run setup

Data Requirements

Input File: sample.txt

Format: JSON array of FDA drug label records

Required Fields:

```
{
  "id": "unique_identifier",
  "openfda": {
    "brand_name": ["Product Name"]
  },
  "indications_and_usage": ["text..."],
  "warnings_and_cautions": ["text..."],
  "dosage_and_administration": ["text..."]
  // ... other sections
}
```

Optional Fields: All section fields are optional. Only sections with content are indexed.

Performance Considerations

Processing Time:

- ~0.5-1 second per record
- Embedding generation is the bottleneck
- Scales linearly with data size

Resource Usage:

- CPU: Moderate (embedding generation)
- RAM: ~2-4 GB (model loading)
- Disk: ~5-10 MB per 100 documents

Optimization Tips:

- Batch process in chunks for large datasets
- Use GPU if available for faster embeddings

- Monitor memory usage for very large corpora
-

Usage Guide

Starting the Application

Step 1: Activate Environment

```
# If using virtual environment
source venv/bin/activate # macOS/Linux
venv\Scripts\activate   # Windows
```

Step 2: Start Ollama (if not running)

```
# Check if Ollama is running
ollama list
```

```
# Start Ollama service (if needed)
ollama serve
```

Step 3: Launch Application

```
streamlit run app.py
```

Expected Output:

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`
Network URL: `http://192.168.1.x:8501`

Step 4: Access Interface

Open browser and navigate to: **`http://localhost:8501`**

User Workflow

Initial Setup

1. **First Run:** Wait for automatic ingestion (2-5 minutes)

- System detects empty database
 - Processes sample.txt
 - Stores in ChromaDB
 - Shows progress in console
2. **Welcome Screen:** See welcome message and product dropdown

Selecting a Medication

1. Click dropdown: "Choose a drug from the database"
2. Select medication from list (e.g., "Aspirin")
3. Confirmation message appears with available commands

Asking Questions

Example Questions:

"What is this medication used for?"
"What are the side effects?"
"Can I take this while pregnant?"
"What are the drug interactions?"
"How should I take this medication?"
"Are there any warnings?"
"What should I do if I miss a dose?"
"Can children take this?"

Response Format:

- Answer based on drug label
- Clear source indication
- "Information not available" if not in label
- No medical advice provided

Generating Reminders

Method 1: Button

1. Click "Generate Reminder" button
2. Wait for processing
3. View JSON schedule in chat

Method 2: Command

1. Type "reminder" in chat
2. Press Enter
3. View generated schedule

Example Reminder Output:

```
{
  "product": "Aspirin",
  "schedule": [
    {
      "time": "Morning (8:00 AM)",
      "instruction": "Take 1 tablet (81mg) with food"
    },
    {
      "time": "Evening (8:00 PM)",
      "instruction": "Take 1 tablet (81mg) with water"
    }
  ],
  "notes": "Do not exceed 2 tablets per day. Take with food to reduce stomach upset."
}
```

Managing Chats

Create New Chat:

1. Click "New Chat" in sidebar
2. New chat appears as "Chat 2", "Chat 3", etc.
3. Start fresh conversation

Switch Between Chats:

1. Click chat name in sidebar
2. View that chat's history
3. Product selection preserved per chat

Reset Chat:

- Click "Quit Chat" button, OR
- Type `/quit` command
- Clears messages and product selection

Change Medication:

- Click "Change Drug" button, OR
- Type `/change` command
- Returns to product selection

Command Reference

Command	Description	Usage
<code>reminder</code>	Generate medication schedule	Type in chat

/change	Change selected medication	Type in chat
/quit	Reset current chat	Type in chat

Best Practices

✓ Do's

- **Ask specific questions** for best results
- **Review complete label** for comprehensive info
- **Use reminders** as supplementary tools
- **Verify critical information** with healthcare provider
- **Keep questions focused** on one topic at a time
- **Report issues** if responses seem incorrect

✗ Don'ts

- **Don't use for emergencies** - Call 911 instead
- **Don't rely solely** on chatbot for medical decisions
- **Don't input personal** health information
- **Don't expect diagnoses** or treatment advice
- **Don't share** medication with others
- **Don't adjust dosages** without doctor approval

Example Use Cases

Use Case 1: New Prescription

User: Selects "Lisinopril"

User: "What is this medication for?"

Bot: [Explains indications from label]

User: "What are the side effects?"

Bot: [Lists adverse reactions]

User: "reminder"

Bot: [Generates dosing schedule]

Use Case 2: Drug Interaction Check

User: Selects "Warfarin"

User: "What medications interact with this?"

Bot: [Lists drug interactions from label]

User: "Can I take aspirin with this?"

Bot: [Provides specific interaction information]

Use Case 3: Pregnancy Information

User: Selects "Ibuprofen"

User: "Is this safe during pregnancy?"

Bot: [Provides pregnancy category and warnings]

User: "What about breastfeeding?"

Bot: [Shares lactation information if available]

Troubleshooting Common Issues

Issue: "No medications found in database"

Solution:

1. Check that sample.txt exists
2. Verify JSON format is correct
3. Restart app to trigger ingestion
4. Check console for error messages

Issue: Slow response times

Solution:

1. Ensure Ollama is running
2. Check system resources (CPU/RAM)
3. Verify